

|                                   |                                                      |
|-----------------------------------|------------------------------------------------------|
| <b>EXP NO: 03</b><br><b>DATE:</b> | <b>ENTERPRISE DATA PROFILING AND SEMANTIC SEARCH</b> |
|-----------------------------------|------------------------------------------------------|

## AIM

To explore and profile a sample enterprise dataset using Python, Pandas, and DuckDB to identify data quality issues and potential AI features, and to implement semantic search using SentenceTransformers and FAISS for enterprise knowledge retrieval.

## SOFTWARE REQUIREMENTS

Software:

- Google Colab
- Python

Libraries:

- Pandas
- NumPy
- DuckDB
- Sentence-Transformers
- FAISS

## PROCEDURE

3. Open Google Colab and create a new Python notebook.
4. Install the required Python libraries.
5. Import Pandas, NumPy, DuckDB, FAISS, and Sentence-Transformers.
6. Create a sample enterprise product dataset using Pandas.
7. Explore the dataset using head(), shape, dtypes, and describe().
8. Identify missing values, duplicate records, and invalid price values.
9. Identify potential AI feature candidates from the dataset.
10. Connect the Pandas DataFrame to DuckDB and execute SQL queries.
11. Create enterprise knowledge documents related to HR, IT, and finance.
12. Load the Sentence-Transformer model and generate document embeddings.
13. Store the embeddings in a FAISS vector index.
14. Convert user queries into embeddings and retrieve similar documents.
15. Display the semantic search results and save the reports as CSV files.

## PROGRAM

### STEP 1: Install and import libraries

```
!pip -q install duckdb faiss-cpu sentence-transformers
import pandas as pd
import numpy as np
import duckdb
import faiss
from sentence_transformers import SentenceTransformer
```

### STEP 2: Create the enterprise dataset

```
data = {
    "Product_ID": [501, 502, 503, 504, 505, 506, 507, 507],
    "Product_Name": [
        "Laptop", "Mouse", "Keyboard", "Monitor",
        "Printer", "Webcam", "Headset", "Headset"
```

```

    ],
    "Category": [
        "Electronics", "Accessories", "Accessories", "Electronics",
        "Office Supplies", None, "Electronics", "Electronics"
    ],
    "Price": [
        55000, 800, 1200, None,
        9000, 2500, -1500, -1500
    ],
    "Stock": [20, 150, 80, 45, None, 60, 40, 40],
    "Product_Type": [
        "Computing Device",
        "Input Device",
        "Input Device",
        "Display Device",
        "Office Device",
        "Peripheral Device",
        "Audio Device",
        "Audio Device"
    ]
}
df = pd.DataFrame(data)
print("Dataset created successfully!")
display(df)

```

### STEP 3: Explore the dataset

```

print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nFirst 5 rows:")
display(df.head())

```

### STEP 4: Identify missing values

```

missing_report = pd.DataFrame({
    "Missing_Count": df.isnull().sum(),
    "Missing_Percentage": (
        df.isnull().mean() * 100
    ).round(2)
})
display(missing_report)

```

### STEP 5: Identify duplicate records

```

print("Duplicate rows:", df.duplicated().sum())
print("\nDuplicate Product IDs:")
display(df[df["Product_ID"].duplicated(keep=False)])

```

### STEP 6: Identify invalid price values

```

invalid_price = df[df["Price"] <= 0]
print("Invalid price records:")

```

```
display(invalid_price)
```

### **STEP 7: Identify AI feature candidates**

```
feature_candidates = pd.DataFrame({  
    "Feature": [  
        "Category",  
        "Price",  
        "Stock",  
        "Product_Type"  
    ],  
    "Possible_AI_Use": [  
        "Category classification",  
        "Price prediction",  
        "Inventory demand analysis",  
        "Product type classification"  
    ]  
})  
display(feature_candidates)
```

### **STEP 8: Perform SQL analysis using DuckDB**

```
con = duckdb.connect()  
con.register("products", df)  
print("DuckDB connected successfully!")
```

```
result = con.execute("""  
    SELECT * FROM products  
    WHERE Category = 'Electronics'  
""").fetchdf()  
display(result)
```

```
result = con.execute("""  
    SELECT  
        Category,  
        AVG(Price) AS Average_Price,  
        COUNT(*) AS Product_Count  
    FROM products  
    GROUP BY Category  
""").fetchdf()  
print("\nAverage Price by Category:")  
display(result)
```

```
result = con.execute("""  
    SELECT * FROM products  
    WHERE Category IS NULL  
        OR Price IS NULL  
        OR Stock IS NULL  
""").fetchdf()  
display(result)
```

### **STEP 9: Create enterprise knowledge documents**

```
documents = [
```

```

    "Employees can apply for leave through the HR portal.",
    "The company provides health insurance benefits to eligible employees.",
    "Employees must change their company passwords regularly.",
    "The IT helpdesk handles computer and network problems.",
    "Employees can download their monthly salary slips from the portal.",
    "New employees must complete the onboarding process.",
    "The finance department manages budgets and expense reimbursements.",
    "Employees should contact HR for questions about leave and attendance."
]
print("Knowledge base created!")
print("Total documents:", len(documents))

```

#### STEP 10: Generate document embeddings

```

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
document_embeddings = model.encode(
    documents,
    convert_to_numpy=True,
    normalize_embeddings=True
)
print("Embedding shape:", document_embeddings.shape)

```

#### STEP 11: Store embeddings in FAISS

```

dimension = document_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(document_embeddings.astype("float32"))
print("Vectors stored:", index.ntotal)

```

#### STEP 12: Perform semantic search

```

def semantic_search(query, top_k=3):
    query_embedding = model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")
    scores, indices = index.search(
        query_embedding, top_k
    )
    print("Search Query:", query)
    print("\nRelevant Documents:\n")
    for rank, (score, idx) in enumerate(
        zip(scores[0], indices[0]), start=1
    ):
        print(f"Rank {rank}")
        print(f"Similarity Score: {score:.4f}")
        print(documents[idx])
        print("-" * 50)

semantic_search("How can I apply for leave?")

```

#### STEP 13: Save the reports

```

missing_report.to_csv(
    "data_quality_report.csv",
    index=True
)

knowledge_df = pd.DataFrame({
    "Document_ID": range(1, len(documents) + 1),
    "Document": documents
})
knowledge_df.to_csv(
    "knowledge_documents.csv",
    index=False
)
print("Reports saved successfully!")

```

## OUTPUT

### Output 1: All libraries imported successfully!

All required libraries were imported successfully.

### Output 2: Dataset created successfully!

Number of rows and columns: (8, 6)

| Product_ID | Product_Name | Category        | Price   | Stock | Product_Type      |
|------------|--------------|-----------------|---------|-------|-------------------|
| 501        | Laptop       | Electronics     | 55000.0 | 20.0  | Computing Device  |
| 502        | Mouse        | Accessories     | 800.0   | 150.0 | Input Device      |
| 503        | Keyboard     | Accessories     | 1200.0  | 80.0  | Input Device      |
| 504        | Monitor      | Electronics     | NaN     | 45.0  | Display Device    |
| 505        | Printer      | Office Supplies | 9000.0  | NaN   | Office Device     |
| 506        | Webcam       | None            | 2500.0  | 60.0  | Peripheral Device |
| 507        | Headset      | Electronics     | -1500.0 | 40.0  | Audio Device      |
| 507        | Headset      | Electronics     | -1500.0 | 40.0  | Audio Device      |

### Output 3: Dataset exploration

Column names: Product\_ID, Product\_Name, Category, Price, Stock, Product\_Type

| Column       | Data Type |
|--------------|-----------|
| Product_ID   | int64     |
| Product_Name | object    |
| Category     | object    |
| Price        | float64   |
| Stock        | float64   |
| Product_Type | object    |

### Output 4: Missing-value report

| Column     | Missing Count | Missing Percentage |
|------------|---------------|--------------------|
| Product_ID | 0             | 0.0                |

| Column       | Missing Count | Missing Percentage |
|--------------|---------------|--------------------|
| Product_Name | 0             | 0.0                |
| Category     | 1             | 12.5               |
| Price        | 1             | 12.5               |
| Stock        | 1             | 12.5               |
| Product_Type | 0             | 0.0                |

#### Output 5: Duplicate records

Duplicate rows: 1

| Product_ID | Product_Name | Category    | Price   | Stock | Product_Type |
|------------|--------------|-------------|---------|-------|--------------|
| 507        | Headset      | Electronics | -1500.0 | 40.0  | Audio Device |
| 507        | Headset      | Electronics | -1500.0 | 40.0  | Audio Device |

#### Output 6: Invalid price records

| Product_ID | Product_Name | Category    | Price   | Stock | Product_Type |
|------------|--------------|-------------|---------|-------|--------------|
| 507        | Headset      | Electronics | -1500.0 | 40.0  | Audio Device |
| 507        | Headset      | Electronics | -1500.0 | 40.0  | Audio Device |

#### Output 7: Potential AI feature candidates

| Feature      | Possible AI Use             |
|--------------|-----------------------------|
| Category     | Category classification     |
| Price        | Price prediction            |
| Stock        | Inventory demand analysis   |
| Product_Type | Product type classification |

### OUTPUT – DuckDB Analysis

#### Output 8

DuckDB connected successfully!

#### Output 9: Products in Electronics category

| Product_ID | Product_Name | Category    | Price   | Stock | Product_Type     |
|------------|--------------|-------------|---------|-------|------------------|
| 501        | Laptop       | Electronics | 55000.0 | 20.0  | Computing Device |
| 504        | Monitor      | Electronics | NaN     | 45.0  | Display Device   |
| 507        | Headset      | Electronics | -1500.0 | 40.0  | Audio Device     |
| 507        | Headset      | Electronics | -1500.0 | 40.0  | Audio Device     |

#### Output 10: Average price by category

| Category        | Average Price | Product Count |
|-----------------|---------------|---------------|
| Electronics     | 17333.333333  | 4             |
| Accessories     | 1000.000000   | 2             |
| Office Supplies | 9000.000000   | 1             |
| None            | 2500.000000   | 1             |

**Output 11: Records with missing values**

| Product_ID | Product_Name | Category        | Price  | Stock | Product_Type      |
|------------|--------------|-----------------|--------|-------|-------------------|
| 504        | Monitor      | Electronics     | NaN    | 45.0  | Display Device    |
| 505        | Printer      | Office Supplies | 9000.0 | NaN   | Office Device     |
| 506        | Webcam       | None            | 2500.0 | 60.0  | Peripheral Device |

**Output 12: Knowledge base**

Knowledge base created!  
Total documents: 8

**Output 13: Embedding shape**

Embedding shape: (8, 384)

**Output 14: FAISS vector storage**

Vectors stored: 8

**Output 15: Reports**

Reports saved successfully!

[ Insert Screenshot 1 here ]

[ Insert Screenshot 2 here ]

[ Insert Screenshot 3 here ]

## **RESULT**

The enterprise product dataset was successfully profiled using Pandas and DuckDB. Data-quality issues such as missing values, duplicate records, and invalid price values were identified. Potential AI feature candidates were determined. Semantic search was implemented using Sentence-Transformers and FAISS to retrieve relevant enterprise knowledge documents.